Pràctica Introducció a l'Aprenentatge Automàtic: Predicció d'esquizofrènia resistent al tractament

Desembre 2025 - Grau en Intel·ligència Artificial

Irene Miralles

Índex

1. Introducció **3**
2. Preprocessament **4**
3. Preparació de variables **8**
4. Modelatge **11**
4.1 Support Vector Machines (SVM) 11
4.2 XGBoost 11
4.3 Regressió logística 11
5. Selecció del model **12**
6. Model Card **13**
7. Bonus 1: Model EBM **14**
8. Bonus 2: Model Random Forest **15**
9. Conclusions **16**
10. Bibliografia **17**

1. Introducció

Aquesta pràctica té com a objectiu desenvolupar un model de classificació capaç de predir si un pacient presenta esquizofrènia resistent al tractament (TRS) o esquizofrènia no resistent al tractament (NTRS) a partir de les variables proporcionades. El treball segueix el flux habitual d'un projecte d'aprenentatge automàtic: preprocessament, preparació de variables, entrenament i comparació de models, selecció del model final i documentació.

Per assolir aquest objectiu s'han desenvolupat i avaluat els següents models: SVM, XGBoost, Regressió Logística customitzada, EBM i Random Forest.

2. Preprocessament

Abans d'entrar en el detall, cal remarcar que el preprocessament es fa dues vegades amb objectius diferents: (1) una primera anàlisi exploratòria per entendre el dataset i detectar possibles problemes (missings, outliers, desbalanceig), i (2) la definició del preprocessament en forma de pipeline quan es fa modelatge i *cross-validation*, de manera que totes les transformacions s'ajusten només amb el train de cada partició i s'evita qualsevol risc de *data leakage*.

Anàlisi estadístic de variables

El dataset està format per 2 variables categòriques, 6 variables binàries i 18 variables numèriques. Les variables binàries i categòriques són les següents:

- **Variables categòriques:** Ethnicity, CYP2D6_metabolic_phenotype
- **Variables binàries:** Sex, Family_history, HLA_DRB1_04_02, HLA_B_15_02, HLA_A_31_01, Del_22q11_2_status

Pel que fa a l'anàlisi estadístic de les variables numèriques destaquem:

- **Duration_untreated_psychosis:** Presenta 8872 valors disponibles (per tant, hi ha *missings*), amb mitjana 19.22 i desviació estàndard 19.55. Els percentils són P25 = 6.4, mediana = 12.5, P75 = 24.3, amb un rang que va de 0.3 fins a 125.0, cosa que suggereix una distribució amb cua dreta i possibles valors elevats puntuals.

- **Polygenic_risk_score:** Pràcticament complet (8999 valors disponibles), amb mitjana 0.0298 i desviació estàndard 0.1404. Els percentils són P25 = -0.067, mediana = 0.02, P75 = 0.114, i el rang és de -0.437 a 0.575, indicant una variable centrada prop de 0 però amb variabilitat apreciable

Estudi de balanceig de la classe objectiu

S'observa un desbalanceig important de la variable objectiu (TRS): al train hi ha 6162 mostres de la classe 0 i 2838 de la classe 1, és a dir, aproximadament 68.47% vs 31.53%. Aquest desbalanceig es pot apreciar visualment en el següent gràfic de barres.



Figura 1: Distribució de la variable objectiu

Davant d'aquest desbalanceig, s'han pres les següents decisions:

- Particionament estratificat (i també estratificació a cross-validation) per mantenir la proporció de classes en totes les particions.
- Selecció de mètriques macro, en especial f1_macro, perquè pondera igual les dues classes i evita que un model "bo" només per a la classe majoritària aparegui artificialment competitiu. Les mètriques macro es calculen per classe i després se'n fa la mitjana simple (cada classe pesa igual, independentment de quants exemples tingui). Això evita que el rendiment sobre la classe majoritària domini el resultat global. Es prioritza f1 perquè combina *precision* i *recall* en una sola mesura: penalitza tant predir massa positius com deixar-se positius sense detectar.
- Quan el model ho permet, ús de class_weight/pesos de classe per penalitzar més els errors en la classe minoritària. D'aquesta forma el model compensarà el desbalanceig de classes.

Identificació de missings i outliers

S'ha identificat presència de valors absents en un subconjunt de variables numèriques, amb els comptatges següents:

Alkaline	Glucose	Triglyce	Lympho	Neutrop	Duration	IL_17A	Polygen

_phosphatase		rids	cyte_count	hil_count	_untreated_psychosis		ic_risk_score
2938	2619	2453	1991	1985	128	1	1

Veient el nombre de *missings* s'ha decidit imputar utilitzant la mediana. Aquesta estratègia és més robusta davant d'*outliers.*
Pel que fa als outliers, s'ha utilitzat el criteri de l'IQR per identificar variables amb valors extrems i quantificar-ne el percentatge.

En el següent gràfic es pot observar quines són les variables amb major presència d'outliers i el seu respectiu percentatge.

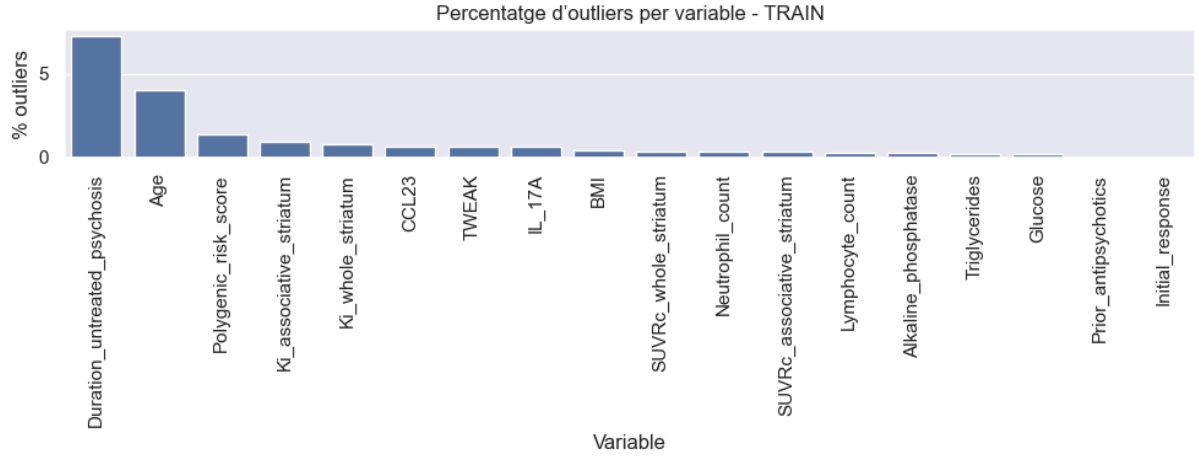


Figura 2: Percentatge d'outliers per variable

Com a decisió de preprocessament base, no s'eliminen ni es corregeixen outliers d'entrada ja que aquests podrien contenir informació clínica rellevant. En la secció de modelatge es prova un tractament d'outliers per al model SVM, però no s'observen millores i, per tant, no es consolida com a pas final.

Recodificació de variables

Les variables categòriques es recodifiquen amb OneHotEncoder per poder-les incorporar als models, ja que la majoria d'algoritmes utilitzats requereixen entrades numèriques i no poden operar directament amb etiquetes categòriques. Aquesta recodificació s'integra posteriorment dins el pipeline per mantenir consistència entre train/test i entre folds de CV.

Particionament del dataset

El dataset es particiona en train/test de forma estratificada per avaluar generalització sobre dades no vistes mentre preservem la proporció de classes en ambdós conjunts.

A partir d'aquest punt, totes les transformacions (imputació, escalat i recodificació) es defineixen perquè s'ajustin només amb el train (i dins de CV, només amb el train de cada fold), minimitzant el risc de data leakage. La recodificació es realitza en el preprocessament de l'apartat de modelatge, mentre que la imputació de *missings* es realitza a continuació i la normalització de variables en la primera cel·la de la secció 2. Això es deu a que aquests dos punts del preprocessament eren necessaris per processos com el PCA o el càlcul de correlacions.

3. Preparació de variables

Normalització de variables

Les variables numèriques es normalitzen mitjançant StandardScaler. Aquesta normalitzaci o s'aplica tant en aquest punt com en la *pipeline* de preprocessament final.

Anàlisi de variables categòriques i variable objectiu

Per analitzar la relació entre la variable objectiu (TRS) i les variables no numèriques, s'han generat gràfics de freqüències tant per a les variables categòriques com per a les variables binàries. Com que el nombre de gràfics és elevat, incloem únicament els de les variables categòriques, tal com demana l'enunciat, i es resumeix la resta de variables binàries de manera global.

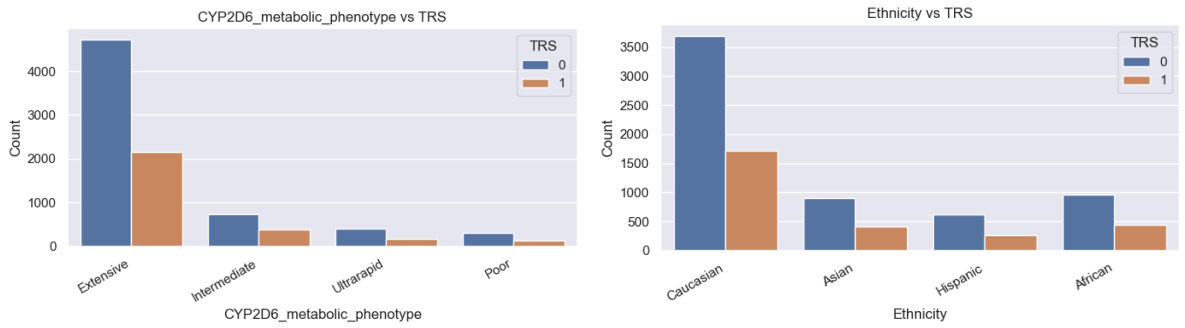


Figura 3 i 4: Gràfics de freqüència variables categòriques vs variable objectiu

En els gràfics podem observar com en *Ethnicity*, la categoria *Caucasian* és majoritària i la resta (*Asian, Hispanic i African*) tenen menys mostres; en totes elles la proporció TRS/NTRS segueix el patró de desbalancejament global i no s'observa una separació clara només per ètnia. De manera similar, a *CYP2D6_metabolic_phenotype* domina el fenotip *Extensive*.

Pel que fa a les variables binàries *Sex* i *Family_history* tenen una presència suficient de 0/1 com per permetre una lectura visual raonable, i en ambdues el recompte de TRS i NTRS segueix el desbalanceig global. D'altra banda, la resta de variables estan molt desbalancejades (la categoria 1 és molt poc freqüent), fet que fa que els gràfics estiguin dominats pel valor 0 i que sigui difícil extreure conclusions robustes només a partir dels comptatges.

Estudi de correlació i eliminació de variables numèriques

Per detectar possibles redundàncies, s'ha realitzat un estudi de correlació entre variables numèriques. Les correlacions més elevades identificades són:

Variable 1	Variable 2	Correlació
Ki_whole_striatum	Ki_associative_striatum	0.9637
Ki_associative_striatum	SUVRc_associative_striatum	0.7314
Ki_whole_striatum	SUVRc_whole_striatum	0.7245
Ki_whole_striatum	SUVRc_associative_striatum	0.7049
Ki_associtive_striatum	SUVRc_whole_striatum	0.701

En la següent matriu de correlació podem apreciar aquestes correlacions de forma visual.

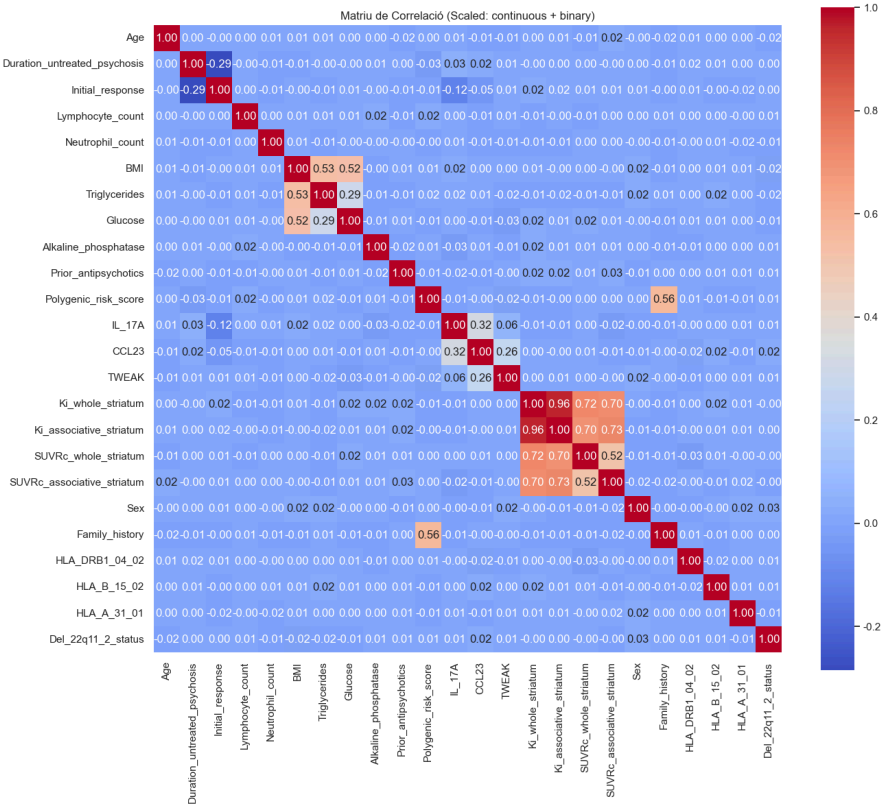


Figura 5: Matriu de correlació

D'acord amb aquest resultat, s'elimina Ki_associative_striatum per reduir redundància, mantenint Ki_whole_striatum com a variable més global, evitant perdre més informació de la necessària.

Estudi de dimensionalitat

S'ha dut a terme un estudi amb PCA per explorar si era viable reduir dimensionalitat amb poca pèrdua d'informació. En aquest estudi s'observa que s'assoleix aproximadament el 95% de variància explicada amb 15 components. El PCA s'utilitza únicament com a eina d'anàlisi ja que s'ha considerat que la seva implementació dificulta la interpretabilitat del model i no aportaria un guany significatiu.

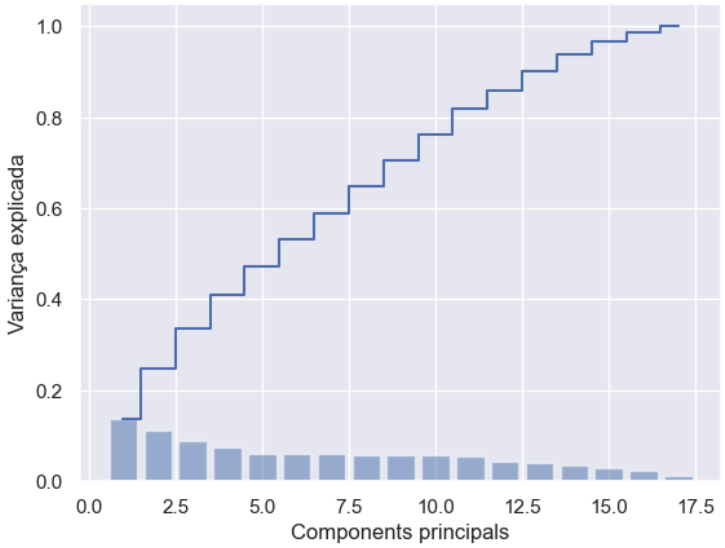


Figura 6: Anàlisi de variança

S'ha considerat que l'alleugeriment del model en eliminar dues variables no era prou rellevant com per justificar la pèrdua d'informació. Per tant, no s'han reduït variables a partir de l'anàlisi de dimensionalitat.

4. Modelatge

A l'apartat de modelatge, tal com s'ha comentat a les seccions anteriors, el preprocessament es defineix dins d'un *pipeline* (amb transformacions separades per a variables numèriques i categòriques) i s'aplica dins la validació creuada, de manera que la imputació de missings, la normalització i la recodificació (OneHotEncoder) s'ajusten sempre només amb el train de cada partició i s'evita qualsevol *data leakage* en l'avaluació dels models.

4.1 Support Vector Machines (SVM)

Anàlisi d'hiperparàmetres

S'ha fet una cerca d'hiperparàmetres amb validació creuada, prioritzant f1_macro com a mètrica de selecció. La cerca s'ha dissenyat per cobrir tant diferents tipus de frontera de decisió (mitjançant el kernel) com el control de complexitat del model i l'efecte del desbalanceig.
S'han analitzat els següents hiperparàmetres amb els següents valors:

- **Kernel (svm__kernel)**
  El kernel defineix la transformació de l'espai d'entrada i, per tant, el tipus de frontera de decisió que pot aprendre el model. S'ha provat el kernel lineal, rbf i poly. D'aquesta forma queden cobertes tant les alternatives lineals com les no lineals.

- **Paràmetre C (svm__C = [0.01, 0.1, 1, 10, 100])**
  L'hiperparàmetre C controla el compromís entre marge ample i penalització d'errors. Un C petit ens proporcionarà una frontera suau, amb menys risc d'*overfitting*, mentre que un C més elevat ajustarà millor el train, però tindrà un risc major de sobregeneralitzar.
  Per aquest motiu hem provat valors que varien des de el 0.01 fins al 100.

- **Paràmetre gamma (svm__gamma = ['scale', 0.001, 0.01, 0.1])**
  Aquest paràmetre només ha estat analitzat per al kernel RBF. Un valor alt implicarà fronteres més suaus, mentre que un valor baix implicarà fronteres més irregulars. Per a escollir els valors s'ha utilitzat un criteri similar al del hiperparàmetre C. S'ha inclòs el valor *scaled* com a valor recomanat per defecte.

- **Grau del polinomi degree (svm__degree = [2, 4])**

Aquest paràmetre només ha estat analitzat per al kernel poly, representa el grau del polinomi i, per tant, la complexitat del model. Graus baixos (2–4) permeten no linealitat moderada mentr que graus alts incrementen molt la capacitat i el risc d'overfitting. Per això s'ha restringit la cerca a 2 i 4, que són valors raonables per captar no linealitats sense fer la cerca inestable.

- **Pesos de classe class_weight (svm__class_weight = [None, 'balanced'])**
  Aquest paràmetre permet ajustar la penalització dels pesos a cada classe. *None* manté els pesos uniformes, mentre que *balanced* equilibra el cost de forma inversament proporcional a la freqüència de la classe, penalitzant més fortament l'error en la classe menys representada. Al tenir un dataset fortament desbalancejat podem esperar que el valor *balanced* doni millor resultat.

La millor combinació trobada per la cerca ha estat: kernel RBF, C = 0.1, gamma = 0.001 i class_weight = 'balanced'.

Validació

Per analitzar el rendiment del model s'ha fet un anàlisi tant numèric com gràfic. Aquest anàlisi s'ha realitzat sota el conjunt de test del split train/test (dades no vistes).

S'han obtingut les següents mètriques.

accuracy	precision _macro	recall_macro	f1_macro	precision (TRS)	recall (TRS)	f1 (TRS)
0.6128	0.5715	0.5776	0.5721	0.4047	0.4824	0.4402

El model obté una accuracy de 0.6128 i un f1_macro de 0.5721, coherents amb precision_macro (0.5715) i recall_macro (0.5776), cosa que indica un rendiment moderat però relativament equilibrat entre classes quan les ponderem igual.

En focalitzar en TRS, s'observa que la precision(TRS) (0.4047) i el recall(TRS) (0.4824) són sensiblement més baixos, amb un f1(TRS) de 0.4402: això suggereix que el model encara comet tant falsos positius (prediu TRS quan no ho és) com falsos negatius (no detecta part dels TRS), fet que queda reflectit a la matriu de confusió. La corba precision–recall també confirma aquesta limitació: la precisió cau quan s'intenta recuperar més TRS (augment de recall).

Finalment, respecte a l'overfitting, el comportament en CV és estable: la f1_macro mitjana en train_cv (0.5924) és molt propera a la de test_cv (0.59), fet que indica que el model no

està memoritzant el train dels folds i que la capacitat està raonablement controlada pels hiperparàmetres seleccionats.

Aquests resultats es poden visualitzar en les següents figures.

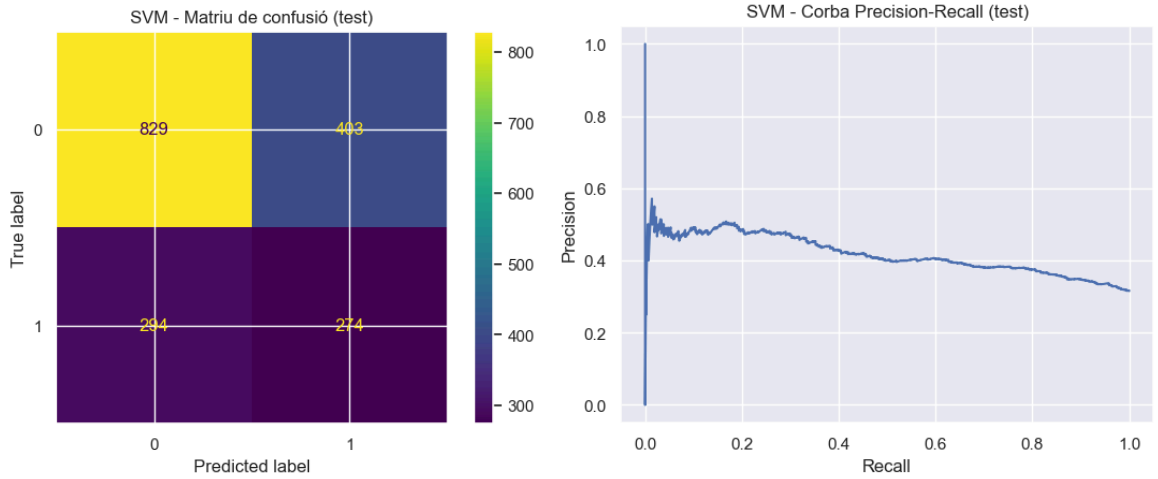


Figura 7 i 8: Matriu de confusió i corba precision - recall del model SVM

Intents de millora

S'han realitzat tres intents de millora diferents.

El primer ha consistit en un reafinament dels paràmetres escollits explorant valors més propers als millors per ajustar millor. Això s'ha aconseguit amb una nova cerca avariciosa sobre nous paràmetres.

El segon intent ha consistit en convertir els outliers de les dues variables més problemàtiques (*Duration_untreated_psychosis* i *Age*) en el valor llindar.
Finalment, el tercer intent ha consistit en una combinació dels dos intents anteriors.

S'han obtingut els següents resultats.

Iteració	f1_macro_train_cv	f1_macro_test_cv	f1_macro_test
Inicial	0.5924	0.59	0.5721
Afinació d' hiperparàmetres	0.5938	0.5923	0.5729
Tractament d'*outliers*	0.585	0.5819	0.5728
Combinació	0.5939	0.5926	0.5721

Els resultats mostren que l'afinació és l'únic intent que aporta una millora consistent, amb f1_macro_test_cv = 0.5923 i el millor f1_macro_test = 0.5729 (lleugerament superior a l'inicial 0.5721). En canvi, el tractament d'outliers redueix el rendiment (f1_macro_test_cv =

0.5819), cosa que suggereix que aquests valors extrems poden contenir senyal útil o que el capping introdueix una pèrdua d'informació. Pel que fa a overfitting, en totes les iteracions la diferència entre f1_macro_train_cv i f1_macro_test_cv és petita, de manera que no s'observa sobreajustament; el criteri de selecció, per tant, es basa en maximitzar el rendiment en test i mantenir estabilitat en CV. Amb aquests criteris, el model escollit dins SVM és el de l'afinació d'hiperparàmetres, ja que obté el millor resultat en test sense penalitzar la generalització.

4.2 XGBoost

Anàlisi d'hiperparàmetres

De nou, per a aquest model també s'ha realitzat una cerca d'hiperparàmetres amb validació creuada, prioritzant f1_macro com a mètrica de selecció. L'anàlisi d'hiperparàmetres té com a objectiu controlar simultàniament: la capacitat del model (profunditat i nombre d'arbres), la velocitat/aprenentatge (learning rate), la robustesa davant soroll i sobreajustament (submostreig i regularització) i la compensació del desbalanceig (pes de la classe positiva).

- **Nombre d'arbres (xgb__n_estimators = [100, 150, 200])**
  Determina quants arbres (iteracions de boosting) s'entrenen. Més arbres incrementen capacitat i poden millorar rendiment, però també augmenten el risc d'overfitting i el cost computacional. Per això s'han provat valors moderats (100–200) que cobreixen des d'un model més lleuger fins a un més complexe.

- **Profunditat (xgb__max_depth = [3, 4, 5, 6, 7, 8])**
  Controla la complexitat de cada arbre. Profunditats baixes generen arbres més simples (menys variància), mentre que profunditats altes permeten capturar interaccions més complexes però poden memoritzar soroll. El rang 3–8 permet explorar des d'un règim conservador fins a un règim més flexible.

- **Taxa d'aprenentatge (xgb__learning_rate = [0.01, 0.05, 0.1])**
  Valors més petits fan passos més curts (millor generalització, però sovint requereixen més arbres), i valors més grans acceleren l'ajust però poden sobreajustar. S'han escollit valors típics en pràctica per cobrir la combinació estabilitat/rapidesa.

- **Submostreig de files (xgb__subsample = [0.6, 0.7, 0.8, 0.9])**
  Percentatge de mostres utilitzades per arbre. Valors < 1 introdueixen aleatorietat i actuen com a regularització (redueixen sobreajustament). El rang 0.6–0.9 permet veure si convé més regularització (0.6) o un entrenament més "complet" (0.9).

- **Submostreig de variables (xgb__colsample_bytree = [0.6, 0.7, 0.8, 0.9])**
  Percentatge de característiques usades per arbre. Igual que subsample, ajuda a reduir la similitud entre arbres i a reduir sobreajustament. S'ha mantingut el mateix rang que en les mostres.

- **Regularització L1 (xgb__reg_alpha = [0, 0.1, 0.5])**

Penalitza valors grans dels pesos, 0 equival a no aplicar L1. Valors més alts incrementen la regularització i poden millorar generalització si hi ha soroll o variables poc informatives.

- **Regularització L2 (xgb__reg_lambda = [0.5, 1.0, 1.5])**
  Penalitza pesos grans de manera suau (estabilitza el model). S'han provat valors al voltant dels habituals (prop de 1) per veure si cal una regularització una mica més baixa o més alta.

- **Pes de la classe positiva (xgb__scale_pos_weight = [1.0, ratio])**
  Ajusta el cost dels errors a la classe minoritària (TRS=1). El valor 1.0 no compensa el desbalanceig mentre que ratio (aprox. NTRS/TRS) força el model a penalitzar més els errors en la classe minoritària.

La millor combinació trobada per la cerca ha estat: n_estimators = 200, max_depth = 6, learning_rate = 0.01, subsample = 0.6, colsample_bytree = 0.6, reg_alpha = 0.5, reg_lambda = 1.5, scale_pos_weight = 2.171806

Validació

De nou, per analitzar el rendiment del model s'ha fet un anàlisi tant numèric com gràfic. Aquest anàlisi s'ha realitzat sota el conjunt de test del split train/test (dades no vistes).

S'han obtingut les següents mètriques.

accuracy	precision_macro	recall_macro	f1_macro	precision (TRS)	recall (TRS)	f1 (TRS)
0.5906	0.5711	0.5813	0.5657	0.3945	0.5563	0.4616

Les mètriques obtingudes mostren una accuracy de 0.5906 i un f1_macro de 0.5657, en línia amb precision_macro (0.5711) i recall_macro (0.5813), fet que indica un rendiment moderat i relativament equilibrat quan ponderem les dues classes per igual.

En canvi, si ens focalitzem en la classe d'interès TRS, s'observa un comportament asimètric: el model aconsegueix recall(TRS) = 0.5563 però amb una precision(TRS) = 0.3945, és a dir, recupera una part apreciable de casos TRS però a costa d'un nombre important de falsos positius; això queda reflectit tant a la matriu de confusió com a la corba precision–recall (Figura 9 i 10).

Finalment, respecte a l'*overfitting*, els resultats de CV indiquen sobreajustament notable, ja que la f1_macro en train_cv (0.7422) és molt superior a la f1_macro en test_cv (0.5797): el model té capacitat per ajustar molt bé els folds d'entrenament però generalitza pitjor del que suggereix el rendiment en train.

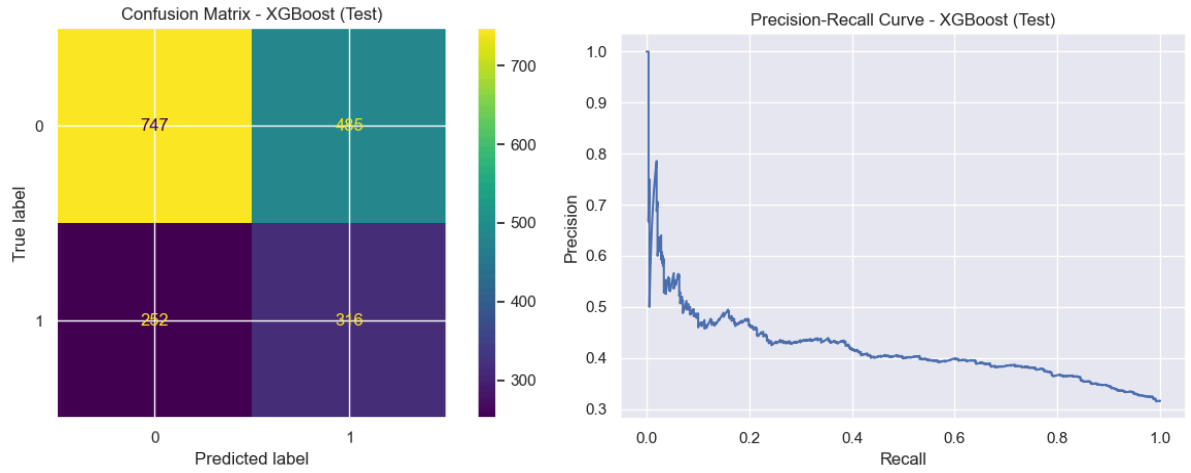


Figura 9 i 10: Matriu de confusió i corba precision - recall del model XGBoost

Intents de millora

En aquest cas només s'ha realitzat un intent de millora que ha consistit en un nou anàlisi d'hiperparàmetres per tal de reduir l'*overfitting* del model anterior.

S'han obtingut els següents resultats:

Iteració	f1_macro_train_cv	f1_macro_test_cv	f1_macro_test
Inicial	0.7422	0.5797	0.5657
Iteració 1	0.641	0.5801	0.5658

Els resultats mostren que la Iteració 1 redueix fortament la f1_macro_train_cv, mentre manté pràcticament igual la f1_macro_test_cv i també el f1_macro_test.

Això és coherent amb un model menys sobreajustat: perd capacitat d'encaixar el train però conserva la generalització. Tot i que la millora és molt petita en mètrica, és preferible quedar-se amb la Iteració 1 dins l'apartat XGBoost ja que generalitza millor sense penalitzar el resultat en test.

4.3 Regressió logística

Creació de la classe

S'ha implementat una classe de regressió logística personalitzada. El model calcula una probabilitat de pertinença a TRS mitjançant la funció sigmoide i s'entrena iterativament amb descens de gradient en mini-batches per minimitzar l'error de classificació. Per tractar el desbalanceig de la variable objectiu, permet activar class_weight='balanced', que incrementa el pes dels errors en la classe minoritària (TRS) durant l'entrenament. A més,

incorpora regularització L2 (paràmetre reg_lambda) per evitar pesos massa grans i millorar la capacitat de generalització.

Anàlisi d'hiperparàmetres

Mantenint l'estil dels models anteriors, aquest cop també s'ha realitzat una una cerca d'hiperparàmetres amb validació creuada, prioritzant f1_macro

- **Mida del batch (lr__batch_size = [4, 8, 16, 32, 64, 128])**
  Determina quantes mostres s'utilitzen a cada actualització dels pesos. Batches petits fan més actualitzacions i poden capturar millor detalls del conjunt d'entrenament, mentre que batches grans fan actualitzacions més suaus i estables però poden quedar-se amb una solució menys òptima per a la validació. S'ha provat un rang ampli per trobar el millor balanç.

- **Learning rate (lr__lr = [0.01, 0.05, 0.1])**
  Controla la mida del pas en cada actualització. Valors petits fan l'aprenentatge més estable però poden requerir més iteracions; valors grans acceleren però poden desestabilitzar o oscil·lar. S'han escollit valors típics i escalats per observar diferències clares de comportament.

- **Regularització L2 (lr__reg_lambda = [0.0, 1e-4, 1e-3])**
  Penalitza pesos grans (de forma suau), reduint el risc d'overfitting quan hi ha soroll o variables poc informatives. S'ha inclòs el cas sense regularització (0.0) i dos valors petits per veure si una penalització lleu ja aporta estabilitat.

- **Nombre d'èpoques (lr__n_epochs = [200, 300, 400])**
  Nombre de passades completes sobre el conjunt d'entrenament. Més èpoques poden millorar l'ajust si el model encara no ha convergit, però també poden incrementar el risc de sobreajustament si el model continua "aprenent" soroll. Es prova un rang moderat per assegurar convergència sense disparar cost computacional.

- **Pesos de classe (lr__class_weight = [None, 'balanced'])**
  Balanced dona més pes a la classe minoritària (TRS) i, per tant, força el model a no "ignorar-la" en un dataset desbalancejat. Era esperable que aquest valor millorés f1_macro, ja que aquesta mètrica pondera les dues classes igual.

La millor combinació trobada per la cerca ha estat: batch_size = 4, lr = 0.01, reg_lambda = 0.001, n_epochs = 200 i class_weight = 'balanced', que és coherent amb un entrenament prudent (learning rate baix), amb regularització activa i amb compensació del desbalanceig.

A més a més, per tal d'analitzar com afecta la mida del *batch* s'ha graficat la seva relació.

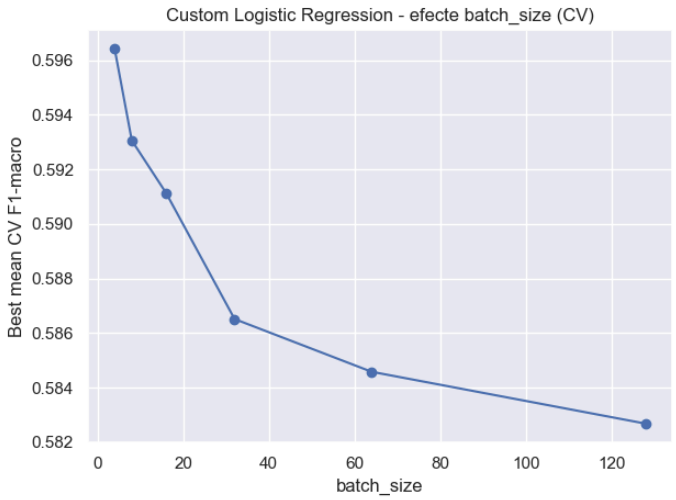


Figura 11: Evolució de la mètrica f1_macro segons la mida del batch

El gràfic mostra una relació clarament decreixent entre batch_size i la mètrica en validació creuada: els valors més petits (especialment batch_size = 4) obtenen el millor f1_macro, i a mesura que el batch augmenta el f1_macro baixa de forma progressiva fins a 128. En conseqüència, dins del rang provat, el millor valor és batch_size = 4, i l'augment del batch size comporta una pèrdua de rendiment.

Validació

De nou, per analitzar el rendiment del model s'ha fet un anàlisi tant numèric com gràfic. Aquest anàlisi s'ha realitzat sota el conjunt de test del split train/test (dades no vistes).

S'han obtingut les següents mètriques.

accuracy	precision _macro	recall_macro	f1_macro	precision (TRS)	recall (TRS)	f1 (TRS)
0.6172	0.579	0.5866	0.5795	0.4127	0.5035	0.4536

El model obté una accuracy de 0.6172 i un f1_macro de 0.5795, en línia amb precision_macro (0.5790) i recall_macro (0.5866), fet que indica un rendiment moderat i força equilibrat quan ponderem les dues classes per igual. Si ens centrem en la classe d'interès (TRS), la precision(TRS) (0.4127) és notablement inferior al recall(TRS) (0.5035), amb un f1(TRS) de 0.4536: això suggereix que el model recupera una part raonable de casos TRS, però a costa d'un nombre elevat de falsos positius (prediccions TRS sobre pacients NTRS). Aquest patró queda reflectit a la matriu de confusió, on hi ha 407 NTRS classificats com a TRS (Figura 12), i també a la corba precision–recall, on la precisió tendeix a disminuir a mesura que augmenta el recall (Figura 13).

Finalment, respecte a l'overfitting, el comportament en validació creuada és estable: f1_macro_train_cv = 0.5967 i f1_macro_test_cv = 0.5964 són pràcticament idèntics, cosa que indica que el rendiment és consistent en validació.

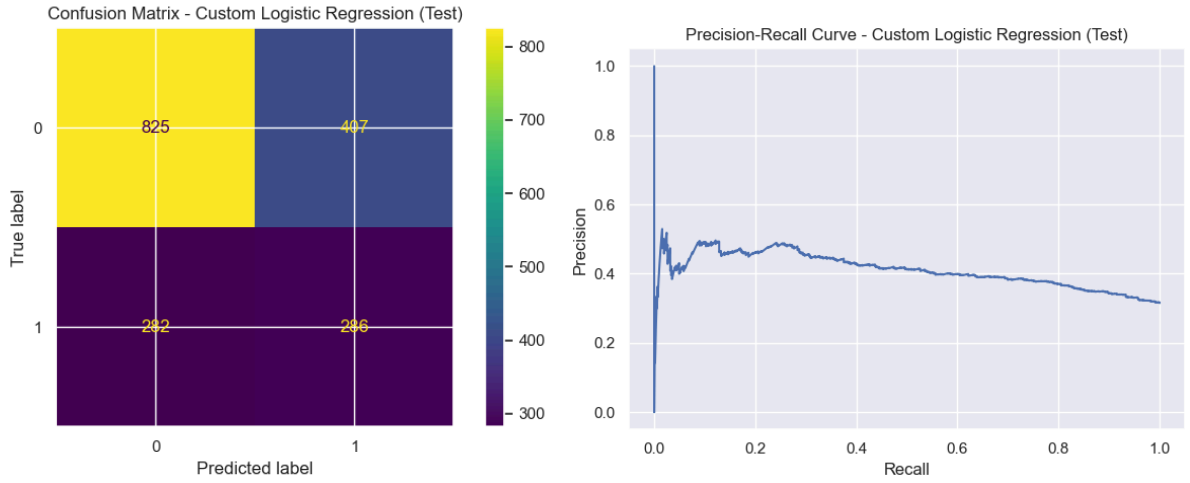


Figura 12 i 13: Matriu de confusió i corba precision - recall del model de Regressió

Intents de millora

En aquest cas també s'ha realitzat només un intent de millora que ha consistit en un nou anàlisi d'hiperparàmetres per tal d'intentar millorar el resultat.

S'han obtingut els següents resultats:

Iteració	f1_macro_train_cv	f1_macro_test_cv	f1_macro_test
Inicial	0.597	0.5964	0.5795
Iteració 1	0.598	0.596	0.5832

Els resultats mostren una millora en el conjunt de test, mentre que en validació creuada el rendiment es manté pràcticament igual i la diferència entre train_cv i test_cv continua sent molt petita, de manera que no s'observa sobreajustament en cap dels dos casos. Amb els mateixos criteris que als models anteriors (prioritzar f1_macro i estabilitat en CV, i usar el test com a comprovació), el model escollit dins la regressió logística és la Iteració 1, ja que aconsegueix el millor resultat en test sense empitjorar el comportament en validació creuada ni introduir senyals d'overfitting.

5. Selecció del model

La selecció del model s'ha basat tant en el rendiment en mètriques com en la capacitat de generalitzar.

A la Taula següent es resumeixen els resultats finals de cada model:

Model	f1_macro_train_cv	f1_macro_test_cv	f1_macro_test
LR (iteració 1)	0.5980	0.596	0.5832
SVM (iteració 1)	0.5938	0.5923	0.5729
XGBoost (iteració 1)	0.641	0.5801	0.5658

Regressió logística custom (Iteració 1) és el model que obté el millor f1_macro_test = 0.5832, i alhora manté una diferència molt petita entre f1_macro_train_cv (0.5980) i f1_macro_test_cv (0.596), no hi ha senyals de sobreajustament.

D'altra banda, SVM (Iteració 1) presenta una validació creuada també estable (0.5938 vs 0.5923), però queda per sota en el conjunt de test (f1_macro_test = 0.5729). Per tant, és una bona alternativa però pitjor segons la mètrica escollida.

Finalment, XGBoost (Iteració 1) tot i assolir un f1_macro_test_cv raonable (0.5801), mostra una diferència més gran respecte a f1_macro_train_cv (0.641), i a més és el pitjor en el test del split (f1_macro_test = 0.5658). En aquest context, això penalitza la seva prioritització, perquè indica més sobreajustament i no compensa en la mètrica final del test.

Per tant, el model seleccionat com a millor és la Regressió logística custom (Iteració 1).

Tot i així, cal remarcar que a la competició de Kaggle el model de regressió logística original obté un resultat lleugerament superior (0.57810 vs 0.57750).

6. Model Card

Per tal de documentar el model final s'ha generat una Model Card que recull la informació essencial del model: què fa, amb quines dades s'ha entrenat i validat, com s'ha avaluat i quines limitacions té.

La Model Card inclou els elements següents:

- Identificació del model i descripció breu: Indica quin algoritme s'ha seleccionat (regressió logística custom), quin és l'objectiu (classificar TRS vs NTRS) i quin criteri s'ha prioritzat (f1_macro). També inclou una referència al preprocessament utilitzat.

- Limitacions i consideracions d'ús: Especifica quines limitacions i consideracions d'ús és necessari tenir en compte a l'hora de fer ús del model. Indica que el model correspon a un treball fet amb fins educatius per a l'assignatura Introducció a l'Aprenentatge Automàtic i que, per tant, no està preparat per a que se li doni cap tipus d'ús mèdic.

- Distribució de dues variables numèriques.

- Avaluació numèrica (mètriques) i gràfica: Recull la mètrica principal del model (f1_macro) tant pel conjunt de train com pel conjunt de test. També inclou una matriu de confusió i una corba ROC sobre el conjunt de test.

7. Bonus 1 i 2: Model EBM i Model Random Forest

Per a la realització dels dos models s'ha utilitzat el mateix preprocessament que en els models anteriors (pipeline amb transformacions separades per variables numèriques i categòriques, incloent imputació de missings, normalització i recodificació amb OneHotEncoder), aplicat dins de validació creuada per evitar *data leakage*. Igualment, s'ha seguit el mateix patró de modelatge: anàlisi d'hiperparàmetres, validació numèrica i validació gràfica sobre el conjunt de test. En aquest apartat, però, no s'han realitzat intents de millora (iteracions addicionals).

Model	f1_macro_train_cv	f1_macro_test_cv	f1_macro_test
LR (iteració 1)	0.5980	0.596	0.5832
SVM (iteració 1)	0.5938	0.5923	0.5729
XGBoost (iteració 1)	0.641	0.5801	0.5658
EBM (bonus 1)	0.5125	0.4856	0.4823
Random Forest (bonus 2)	0.8333	0.5878	0.5687

Els resultats de la taula mostren clarament que l'EBM té valors baixos tant en train_cv com en test_cv (0.5125 vs 0.4856), cosa que indica que no arriba a capturar prou bé els patrons. En canvi, el Random Forest té una diferència molt gran entre train_cv i test_cv (0.8333 vs 0.5878), és a dir, aprèn massa bé els exemples d'entrenament però després baixa en dades no vistes, un patró típic d'overfitting. Per tot això, tot i haver implementat els dos models cap dels dos millora realment els resultats dels models principals i no es trien com a model final.

8. Conclusions

Conclusions generals

La comparació final mostra que la regressió logística customitzada (Iteració 1) és el model més adequat en aquest context perquè aconsegueix el millor rendiment en test (f1_macro_test = 0.5832) i manté un comportament estable en CV, mentre que alternatives com SVM queden lleugerament per sota i XGBoost presenta senyals més clars de sobreajustament.

Pel que fa als bonus, l'EBM queda lluny en rendiment i el Random Forest, tot i obtenir una puntuació de test raonable, mostra una diferència train_cv–test_cv molt elevada, fet que mostra sobreajustament, a més a més, no supera el model final.

Finalment, en l'elaboració del treball també destaca la importància de:

- Mantenir un preprocessament únic i replicable per evitar inconsistències i possible *data leakage.*
- Utilitzar particionament estratificat i validar amb cross-validation per obtenir una estimació més robusta del rendiment. -
- Saber escollir una mètrica alineada amb l'objectiu (f1_macro) per no prendre decisions esbiaixades pel desbalanceig.

Punts a millorar

Tot i que el model final escollit aconsegueix una predicció raonable encara hi ha diversos punts que es podrien millorar o explorar en treballs futurs.

En primer lloc, caldria aprofundir en una anàlisi de variables rellevants per entendre millor quines característiques aporten valor real a la predicció de TRS.

En segon lloc, es podrien fer més intents de millora centrats en refinar el rendiment sobre la mètrica objectiu, per exemple afinant encara més la cerca d'hiperparàmetres, provant estratègies alternatives de ponderació de classes o provant alternatives de preprocessament.

En tercer lloc, seria convenient analitzar com afecta el desequilibri extrem d'algunes variables binàries al model.

Finalment, en relació amb els Bonus, es podria millorar la implementació mitjançant la realització de intents de millora, així com s'ha fet en els models anteriors. A més a més, en el bonus 2 es podrien explorar altres tècniques d'aprenentatge.

9. Bibliografia

Per a la realització d'aquesta pràctica s'han consultat les següents referències:

1) *LogisticRegression*. (s/f). Scikit-Learn. Recuperado el 28 de diciembre de 2025, de https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
2) *SGDClassifier*. (s/f). Scikit-Learn. Recuperado el 28 de diciembre de 2025, de https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html
3) *SimpleImputer*. (s/f). Scikit-Learn. Recuperado el 28 de diciembre de 2025, de https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html
4) *SVC#*. (s/f). Scikit-Learn. Recuperado el 28 de diciembre de 2025, de https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html?utm_source=chatgpt.com
5) *XGBoost Parameters — xgboost 3.1.1 documentation*. (s/f). Readthedocs.Io. Recuperado el 28 de diciembre de 2025, de https://xgboost.readthedocs.io/en/stable/parameter.html